



ISLAMIC UNIVERSITY OF TECHNOLOGY (IUT)

COURSE CODE: EEE 4308

COURSE NAME: DIGITAL ELECTRONICS LAB

PROJECT: SAP-1 Architecture based 8-bit Computer Design and Implementation.

Phase: B

GROUP NAME: Boolean Squad

GROUP MEMBERS:

1. IKRAM HOSSAIN (230021111)
2. NADMAN IBNE MAMUN (230021114)
3. FARHAN ISHRAQ (230021124)
4. SAMIN YASAR SAMI (230021138)
5. ZAVID ATAHAR SAMIN (230021155)
6. AGHA MAKINUN AMIN CHOWDHURY (230021232)
7. RAZEEN ABRAR RAHMAN (230021215)

OBJECTIVES:

The objective of phase B is to design and implement the following parts:

1. Registers (Accumulator, B Register)
2. Random Access Memory (RAM)
3. Feature Submission

DESCRIPTION:

Registers:

Registers are small, fast memory units inside a CPU. They temporarily store data that the CPU needs immediately for processing. Registers help the CPU work faster because accessing data from registers is much quicker than accessing it from main memory.

Accumulator/ A Register (AC):

The Accumulator or A register is the primary working register in a CPU. It is used to store data temporarily and is the main place where the Arithmetic Logic Unit (ALU) performs operations.

- During a calculation, the ALU usually takes one input from the AC.
- The result of any arithmetic or logic operation is stored back in the AC, making it the main storage for immediate results.
- The AC can also send data to the bus (for memory or I/O communication) or receive data from the bus, depending on the instruction being executed.

In simple terms, the AC is like the CPU's main calculator, holding the numbers you are currently working with and storing the result of every calculation.

B Register:

The B Register is a temporary helper register used mainly for ALU operations.

- When the CPU performs operations like addition or subtraction, the B Register holds the second value needed for the ALU.
- Unlike the AC, the B Register usually does not connect to the bus, so it cannot directly send or receive data from memory or I/O devices.
- Its main role is to support the AC during computation, acting as a temporary storage place for one of the operands.

We can consider the B Register as the assistant of the AC because it holds the number that will be used in a calculation but does not store results permanently or communicate outside the CPU.

RAM:

The RAM also known as Random Access Memory, is generally the primary volatile memory . But in case of our SAP1 8bit computer it is used to store both program instruction and data.

- This RAM has 16 addresses, each storing 8-bit data.
- The data can be stored or accessed by entering the suitable address.

Random Access Memory(RAM)

BLOCK DIAGRAM:

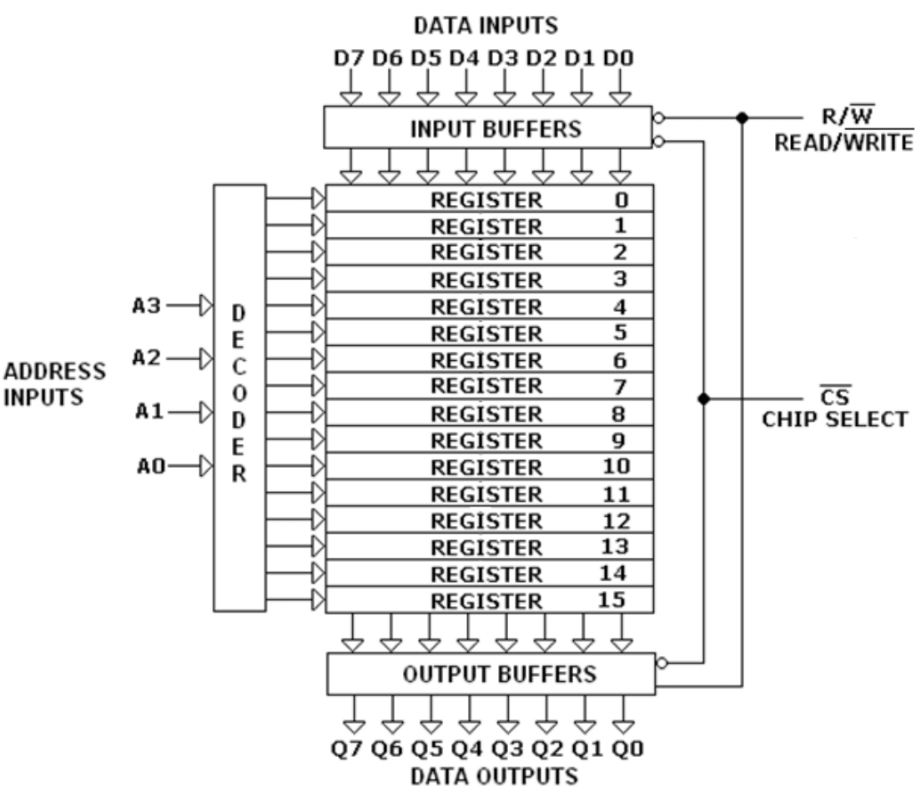


Fig. Blockdiagram of 74LS189

Introduction to 64bit RAM IC (74LS189):

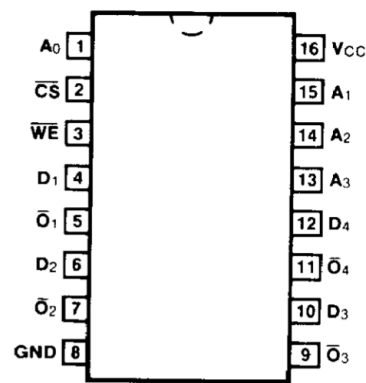


Fig. Pin diagram of 74LS189

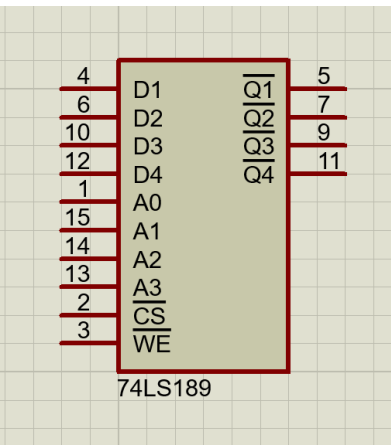


Fig. Schematic of 74LS189 in Proteus

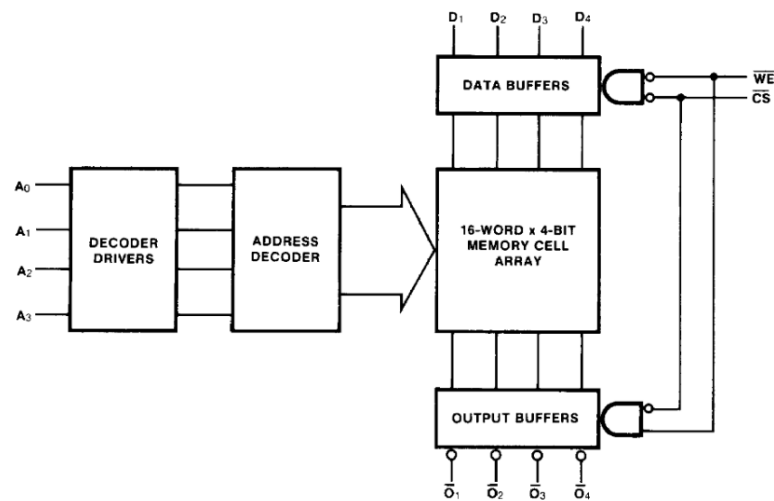


Fig. Logic diagram of 74LS189

The IC used in the design of the RAM is 74LS189. It is a 64bit static RAM IC that has in total 16 pins which are used as

- 4 address pin (A₀, A₁, A₂, A₃),
- 4bit data input (D₁, D₂, D₃, D₄)
- 4 inverted output of the stored data (Q₁', Q₂', Q₃', Q₄')

Additionally there are $\overline{WE}$ '(Write Enable) and $\overline{CS}$ '(Chip Select), where both of the are active low, and at last Ground and Power pin.

Function Table:

Input		Operation	Condition of Output
CS'	WE'		
0	0	Write	High impedance
0	1	Read	Complement of stored data
1	X	Inhibit	High impedance

CIRCUIT DIAGRAM:

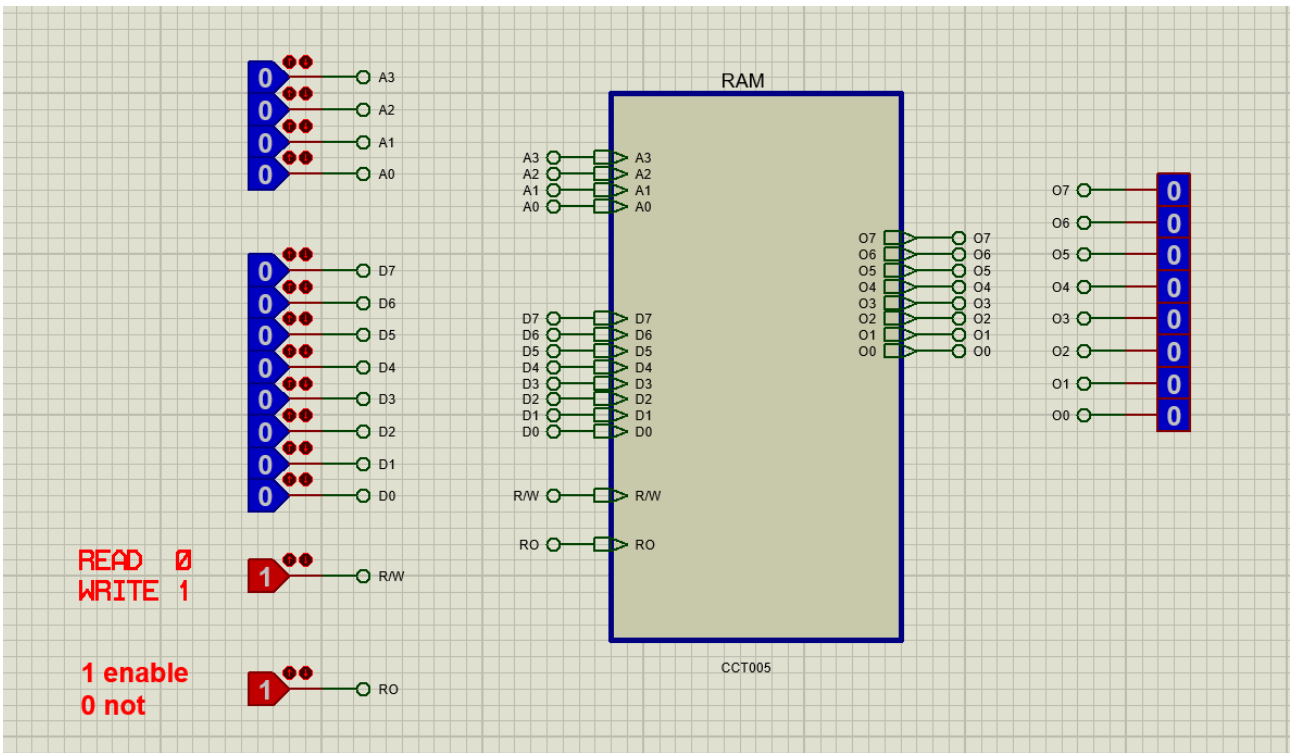


Fig. Subcircuit diagram of RAM

The pin in this circuit is similar to the one explained in the introduction section of the IC. There are two additional pins; R/W and RO.

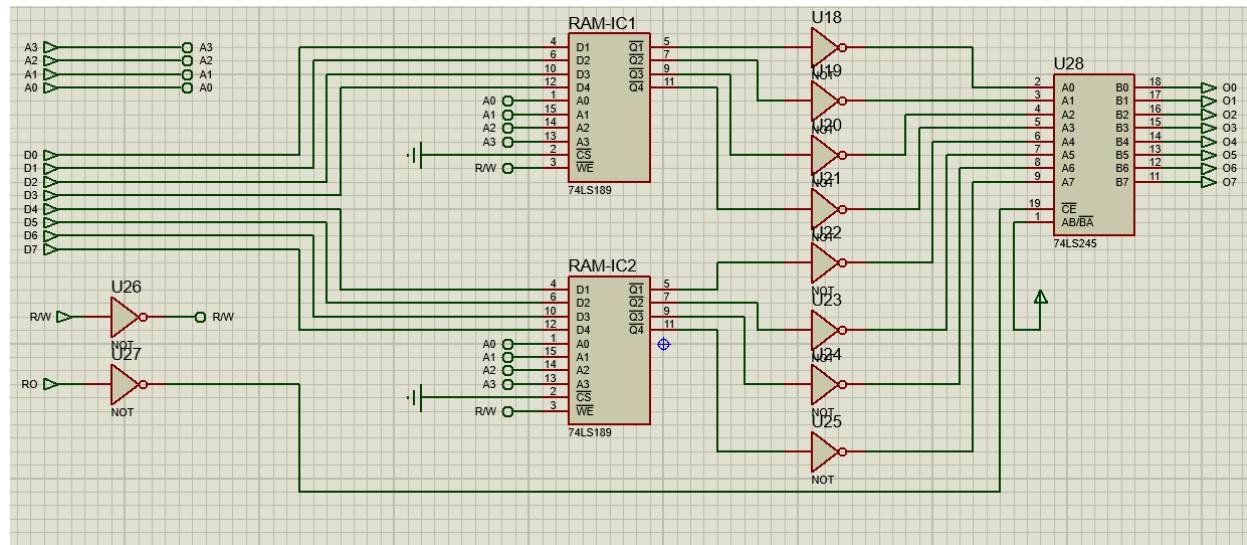


Fig. Childsheet circuit diagram of RAM

As we need to build a 16X8 bit RAM, we used two 64bit 74LS189 RAM ICs. The address pin in both of the ICs where shorted with the same name and input is given from one address. A we have 8 data bit each IC is responsible for 4bit of data. Again the each outputs are passed through a Inverter to get the actual stored data bit. At last the 8bit data from the IC is passed through a Buffer IC which is controlled by RO input.

Algorithm of the RAM:

The RAM designed in this project follows some instruction that is used to operate the RAM. They are given below,

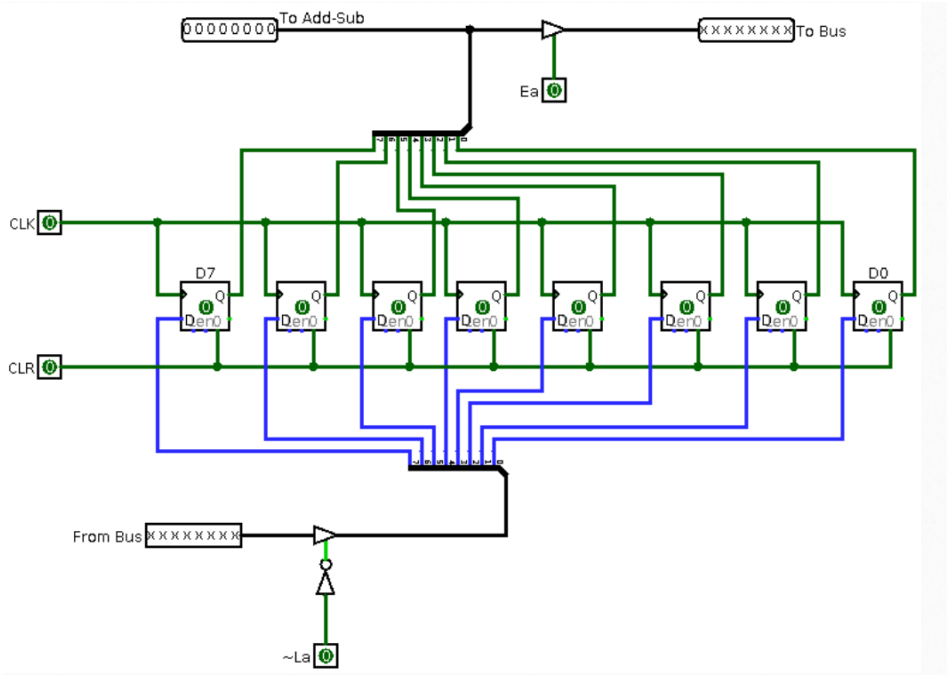
- R/W - selects if the IC is in read mode or write mode.
0- IC is in Read mode.
1- IC is in Write mode.
- When R/W=0, then given 8bit data will be stored at the entered Address
When R/W=1, then stored data will be passed to output pin.
- RO - RAM Out, indicates if the data in the output pin of the RAM would be passed to the bus or not.
0- Data dosen't get passed to bus.
1- Data get passed to bus.

Equipment / Component List:

SL	Component Name	Quantity	Purpose in the Circuit
1	74LS189 (64-bit RAM)	2	These are labeled RAM-IC1 and RAM-IC2 in the diagram. They store the bits of data.
2	74LS245 (Buffer IC)	1	Labeled U28 . This acts as the bridge to send data back to the bus (b1-b8).
3	NOT Gate	10	Used where the input and outputs are needed to be inverted as our desire.
4	Logic States	14	These are the red/blue toggles for R/W, RO, 4 Address bit(A0-A1) and the 8 Data bits (D1-D8) .
5	Logic Probes	8	These are used to see the 8 output (O0-O7) of the RAM.

ACCMULATOR / A Register (AC)

BLOCK DIAGRAM:



Function Table:

State	Reset	AI (In)	AO (Out)	Clock	Action
Reset	1	X	X	X	Clear data to 0000 0000
Hold	0	0	0	Trigger	No change; output stays off Bus
Load	0	1	0	Trigger	Save data from Bus
Output	0	0	1	Trigger	Send stored data to Bus

Algorithm of the Accumulator (A-Register)



Function Tables:

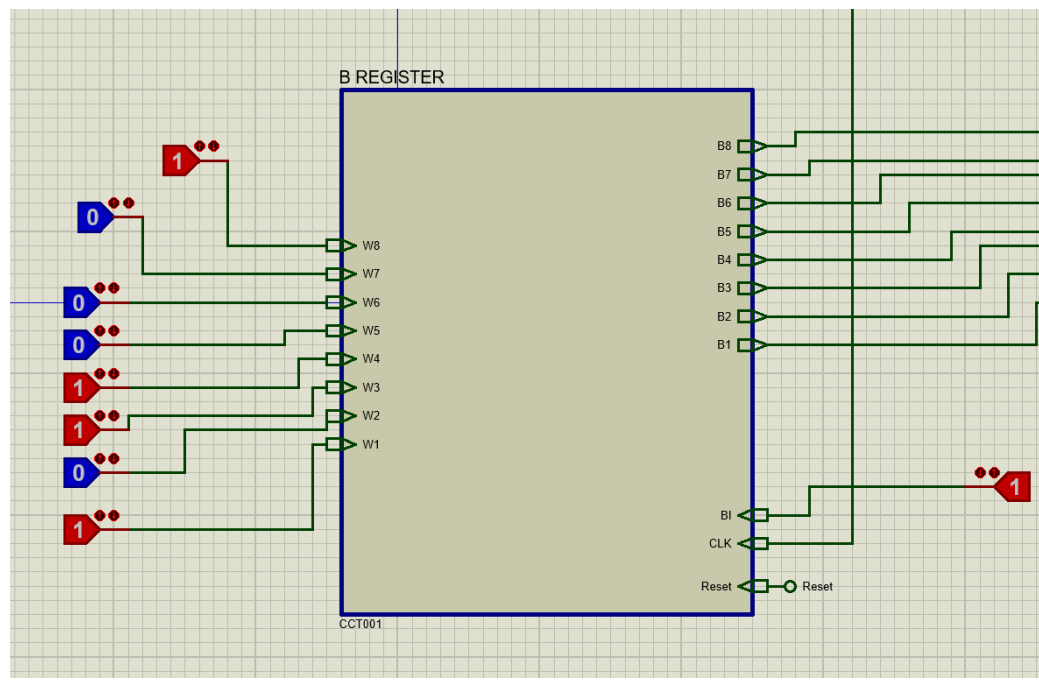
Operation Mode	Reset	B-In (BI)	Clock	Action
System Reset	1	X	X	All 8 bits become 0000 0000 immediately.
Hold Data	0	0	Trigger	No change. Keeps the old data.
Load Data	0	1	Trigger	Reads 8-bit data from the Bus and saves it.

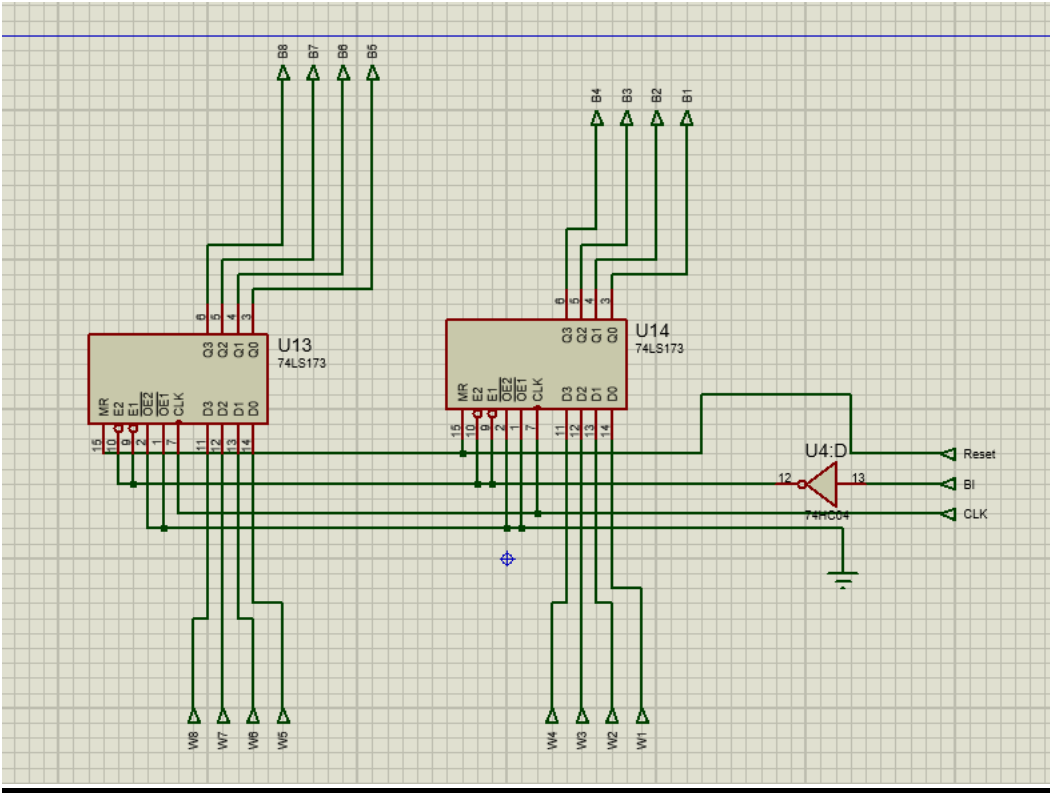
Algorithm of the B-Register

1. **Step 1 (Start):** The register waits for a signal while holding its current 8-bit value.
2. **Step 2 (Check Reset):** If the Reset signal is High (1), the register clears all its internal memory to **0000 0000** instantly. It does not wait for the clock.
3. **Step 3 (Check Control Signal):** If Reset is 0, the register looks at the BI (B-In) signal.
4. **Step 4 (Data Capture):** When BI = 1 and a Clock Trigger occurs, the register "grabs" the 8-bit data currently present on the Data Bus.

5. **Step 5 (Data Output):** The stored 8-bit data is always sent to the ALU (Input B) for calculation. It never goes back to the Bus.
6. **Step 6 (Loop):** The register continues to hold this data until a new BI signal and Clock Trigger arrive, or the Reset is pressed.

Circuit Diagram:





Equipment / Component List:

SL	Component Name	Model	Quantity	Purpose in the Circuit
1	4-bit D-Type Register IC	74LS173	2	Labeled as U13 and U14 . They store the 8-bit data.
2	Hex Inverter (NOT Gate)	74HC04	1	Labeled as U4:D . It inverts BI (B-In) signal.
3	LEDs	LED-AQUA	8	Labeled as D9, D10, D11... D16 to show the 8-bit output (B1-B8).
4	Resistors (330Ω)	330	8	Labeled as R13, R14, R15...R20 to protect the LEDs.
5	Logic States	Switches	11	For BI, CLK, Reset and 8 Data inputs (W1-W8).

Discussion

Outcomes:

1. Register Module

After designing and testing the Accumulator and B-Register, we achieved the following results:

- **Reliable Data Storage:** The registers can successfully "latch" and hold 8 bits of data. Once a number is loaded from the bus, it stays there safely until we either reset it or load a new number.
- **Controlled Data Flow:** By using the AI and BI signals, we have full control over when the registers should listen to the bus. This prevents the registers from picking up "garbage" data or noise from the bus.
- **Successful Bus Interfacing:** In the Accumulator, the 74HCT245 transceiver works perfectly. It keeps the register disconnected from the bus when not in use, which prevents "data collisions" (when two parts of the computer try to talk at the same time).
- **Stable ALU Inputs:** Both registers provide a steady 8-bit signal to the ALU. This is critical because if the input bits flickered or changed unexpectedly, the ALU would give wrong math results.
- **Visual Feedback:** The LED bank allows us to instantly verify if the binary data on the bus matches what is actually stored inside the chips.

2. Random Access Memory(RAM)

- **Program Storage:** RAM stores instructions and data required for program execution.
- **Direct CPU Access:** The Program Counter directly addresses RAM locations. Enables fast instruction fetch and data access.
- **Simple and Reliable Operation :** Small memory size keeps the design easy to understand and implement.

Limitations:

1. Register Module:

Every circuit has its limits. Here are the ones we identified for these registers:

- **Loss of Memory on Power-Off:** These registers are "volatile." If you turn off the power for even a second, all the stored data is lost, and the registers will reset to random values or zeros when turned back on.

- **Small Data Size:** Since this is an 8-bit system, the registers can only hold numbers from 0 to 255. If a calculation results in a number larger than 255, these registers cannot store it properly (this is called an overflow).
- **Speed Limits (Propagation Delay):** It takes a few nanoseconds for the data to travel through the NOT gates and into the 74LS173 chips. While this is very fast, it means we cannot increase the "Clock Speed" of the computer beyond a certain point.
- **Manual Management:** Currently, we have to manually flip switches to load or output data. If the user makes a mistake and activates AI and AO at the wrong time, it can cause errors in the data.

2. Random Access Memory (RAM)

- **Very Limited Memory Size:** Only 16×8 of memory that can restricts program length. Even it is suitable for this kind of basic and simple computer architecture like SAP1, there should be a huge amount of change needed to be made so it is suitable for more suitable and complex computer architecture.
- **No Separate Program and Data Memory:** Same RAM stores both instructions and data. Can cause overwriting issues in larger programs.
- **No Memory Protection:** Risk of accidental data corruption.
- **No Addressing Modes:** Only direct addressing is supported. Limits flexibility in data access.
- **No Expand-ability:** Memory size cannot be increased without major hardware changes.

FEATURE

The features we wanted to add are given below:

1. Additional instructions
2. 8-bit binary to BCD display

DESCRIPTION OF THE FEATURES:

a. ADDITIONAL INSTRUCTIONS:

1.MVI (move immediate):

This instruction bypasses the need for a memory address reference, as the operand is the data itself.

- **Opcode Fetch:** Like all SAP-1 instructions, the first three T-states (T_1, T_2, T_3) are dedicated to fetching the instruction from RAM into the instruction register.
- **Instruction Format:** In an 8-bit architecture, the upper 4 bits represent the opcode (e.g., 1001), and the lower 4 bits contain the **immediate data** (0-15 in decimal).

- **Execution (T₄):** The lower 4 bits of the Instruction Register (IR) are enabled onto the W-bus (E₁ signal). Simultaneously, the Accumulator is set to load (L_A signal).
- **Efficiency:** This instruction is faster than LDA because it does not require a second memory access to fetch data from an address; the data is already inside the IR.

2. ANA (Bitwise AND)

This instruction performs a logical AND between the Accumulator (A) and a value stored in RAM.

Memory Access (T₄): The address portion of the IR is sent to the MAR (E₁, L_M), and the data from that RAM location is loaded into the B-Register (E_R, L_B).

Logic Operation (T₅): The ALU performs the bitwise AND operation (A AND B).

- **Result Storage (T₆):** The output of the ALU logic gates is enabled onto the bus and latched back into the Accumulator (E_u, L_A).
- **Hardware:** We have to add 74LS08 (AND gates) to the ALU stage.

3. ORA (Bitwise OR)

This instruction performs a logical OR between the Accumulator and a value stored in RAM.

- **Memory Access (T₄):** Like ANA, the operand address is sent to the MAR, and the byte is fetched from RAM into the B-Register.
- **Logic Operation (T₅):** The ALU performs the bitwise OR operation (A OR B).
- **Result Storage (T₆):** The ALU result is latched into the Accumulator.
- **Hardware:** We have to add 74LS32 (OR gates) to the ALU stage.

Hardware Requirements for Logic Operations

To support ANA and ORA, SAP-1 ALU needs a Multiplexer (like the 74LS151 or 74LS153). This allows the controller to select whether the "Sum" bus should carry the Addition, Subtraction, AND, or OR result based on the current opcode.

2. Binary to BCD display:

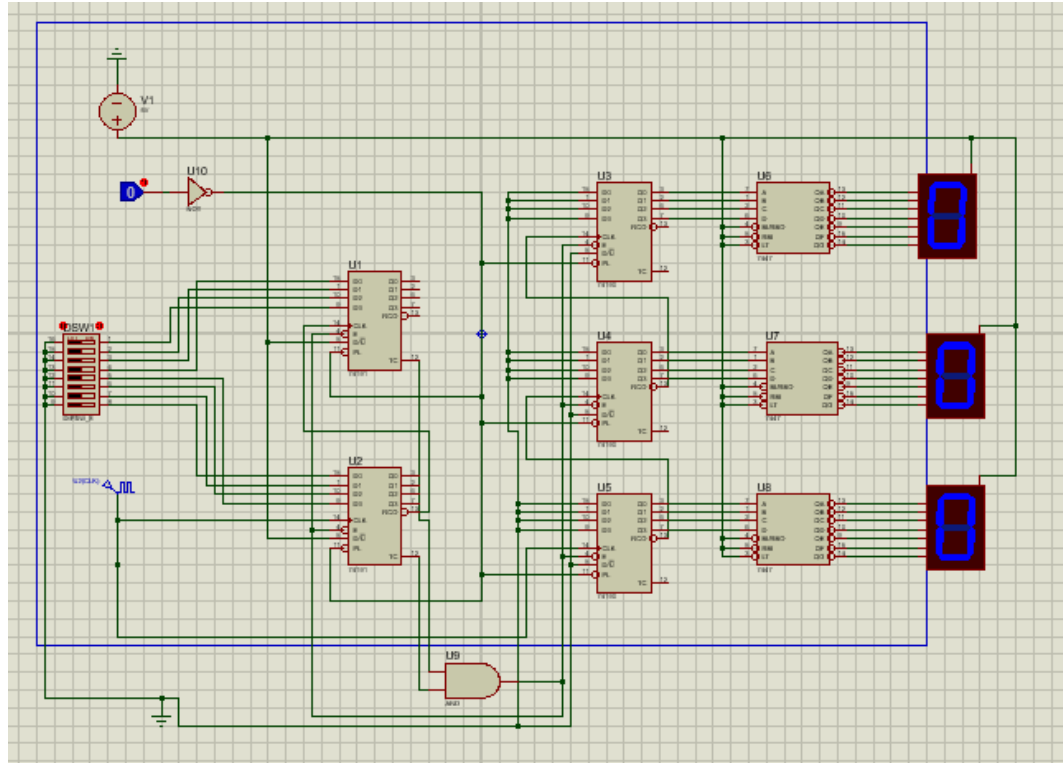


FIG: BINARY TO BCD CONVERTER CIRCUIT

This conversion method can be called synchronous down counter/ up counter binary to BCD converter.

1. Input and Control Stage

- **8-bit DIP Switch (S1):** Sets the initial 8-bit binary value to be converted.
- **Push-button (S2):** Triggers the LOAD signal, preset-loading the binary value into the down-counters and resetting the BCD counters to zero.
- **High-Speed Clock (U3):** Provides the pulses required for the simultaneous counting process.

2. Binary Down-Counter Section (The "Timer")

- **ICs:** Two cascaded **74LS191** 4-bit binary counters.
- **Function:** Configured to count down from the value set by the DIP switches.
- **Stop Logic:** A **7408 AND gate (U4A)** monitors the MAX/MIN or terminal count outputs. When the count reaches zero, the logic level changes to disable the CTEN (count enable) on all counters, freezing the system.

3. BCD Up-Counter Section (The "Accumulator")

- **ICs:** Three cascaded **74LS190** synchronous BCD counters (Units, Tens, Hundreds).
- **Function:** These counters start at 000 and increment by one for every clock pulse.
- **Conversion:** Since they receive the exact same number of clock pulses as the binary counters take to reach zero, the final BCD value stored matches the original binary input.

4. Display and Decoding Section

- **ICs:** Three **7447 BCD-to-7-Segment Decoders**.
- **Output:** Converts the BCD data into signals for three **Common-Anode 7-segment displays**, providing a human-readable decimal representation of the 8-bit binary number (0 to 255).

Operational Specifications

- **Conversion Time:** $T = N/f$ (where N is the decimal value and f is the clock frequency).
- **Clock Frequency:** Initially tested at 1 kHz for visual verification; intended operational frequency is 20 kHz to ensure near-instantaneous display updates.

